ASTRO PI · MISSION A · OPGAVEHÆFTE

Space Gardeners: kan der gro kartofler på ISS?

En guide til at sætte en Astro Pi op i klasselokalet, forstå koden fra et rigtigt vinderprojekt og bygge jeres egen mission til Mission Space Lab 2026/27.

Målgruppe gymnasiet, informatik B/A **Tid** 2 × 45 min + åben opgave

Forudsætninger grundlæggende Python

Indhold

01 Projektet 3

02 Kør den originale kode 4

03 Sådan virker koden 5

04 Fejljagt 7

05 Klar til rummet: 2026-versionen 8

06 Jeres egen mission 10

07 Til læreren 11

A Udstyr og opsætning 12

01

Projektet

I 2017/18 skrev to 3.g-elever, Mohammad Ahmad og Frederik Paustian, fra Nærum Gymnasium et Python-program til Mission Space Lab. Ideen hed **Space Gardeners**: brug sensorerne på Astro Pi'en til at måle, om luften inde på Den Internationale Rumstation har de forhold, en kartoffelplante skal bruge for at gro. Programmet blev udvalgt af ESA og kørte på ISS i maj 2018.

Da holdet gik i gang, havde de slet ingen forstand på, hvad det betød at kode. De lærte Python undervejs, ved at prøve sig frem, google fejlbeskeder og spørge deres lærer. Det kan man se i koden, og det er pointen med dette hæfte: man behøver ikke kunne det hele på forhånd for at sende kode til rummet.

Koden måler fire ting og sammenligner dem med, hvad en kartoffel har det godt med:

Måling	Sensor	Kartoflen trives ved
Temperatur	Sense HAT	15–20 °C
Luftfugtighed	Sense HAT	80–95 %
Lufttryk	Sense HAT	ca. 1013 mbar
Lys	Kamera	middel til højt

I DETTE HÆFTE

I starter med den originale kode fra 2018, så I kan se, hvordan et rigtigt elevprojekt ser ud, fejl og det hele. Bagefter bygger I en version, der overholder reglerne for Mission Space Lab 2026/27, og til sidst formulerer I jeres egen mission.

02

Kør den originale kode

FØR I STARTER

Vi antager, at I har en Raspberry Pi med Sense HAT og kamera, sat op med Raspberry Pi OS og Thonny. Det er det, vi kalder en Astro Pi. Har I ikke det endnu, så start med Appendiks A: Udstyr og opsætning og kom tilbage hertil bagefter. Har I slet ingen hardware, kan dataopsamlingen også testes i Astro Pi Replay Online — https://astro-pi-replay-online.astro-pi.org/.

Koden fra 2018 brugte biblioteket `picamera`, som ikke findes i nyere Raspberry Pi OS. Ret de linjer, der har med kameraet at gøre, så bruger koden `picamzero` i stedet. Resten af koden kan stå, som den er.

Original (2018)	Ret til
`from picamera import PiCamera` (og de to andre picamera-linjer)	`from picamzero import Camera`
`camera = PiCamera()`	`camera = Camera()`
`camera.resolution = (100, 100)`	Slet linjen
`camera.exposure_mode = 'auto'` `camera.shutter_speed = …`	Slet begge linjer
`camera.capture('image.jpg')`	`camera.take_photo('image.jpg')`
`photo.getpixel((50,50))`	`photo.getpixel((width // 2, height // 2))`
`camera.stop_preview()`	Slet linjen

Opgave 1 Få den til at køre

1. Åbn `AstroPi-Kartoffelkode.py` i Thonny og ret kamera-linjerne.
2. Tryk **Run**. Se målingerne komme frem i Thonnys shell og søjlerne på LED-skærmen.
3. Hold hånden over kameraet. Hvad sker der med lys-søjlen og status-teksten?
4. Pust forsigtigt på Sense HAT'en. Hvilken søjle reagerer?
5. Stop programmet med **Stop**-knappen. Åbn filen `team_data.csv`, som er blevet lavet ved siden af koden.

03

Sådan virker koden

Programmet er én stor løkke, der gentager de samme fem trin, indtil man stopper det.

1. Mål temperaturen

```
linje 45-47

t = sense.get_temperature()
t = round(t,1)
t = t - 12   # Sense HAT'en sidder oven på Pi'ens varme CPU
```

Sensoren sidder tæt på processoren og måler for højt. Holdet trak bare 12 grader fra. Det er en grov rettelse, men den virkede på ISS-hardwaren dengang.

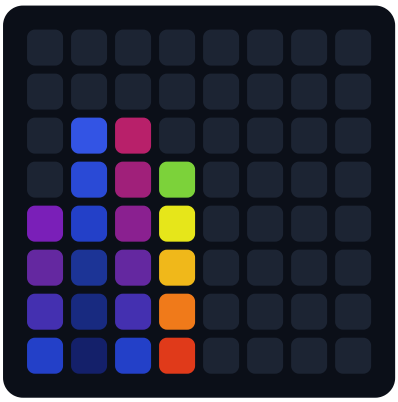
2. Mål lyset med kameraet

```
linje 52-66

camera.capture('image.jpg')
photo = Image.open('image.jpg').convert('RGB')
RGB = photo.getpixel((50,50))     # farven i midten af billedet
R,G,B = RGB
lightvalue = (R + G + B) / 3 / 255 * 100   # 0-100 %
```

Kameraet tager et billede, og koden læser farven på én pixel i midten. Gennemsnittet af rød, grøn og blå, regnet om til procent, bliver programmets lysmåling. Bagefter oversættes tallet til en tekst fra *Very bad* til *Optimal*.

3. Tegn søjler på LED-skærmen



- Kolonne 0: temperatur
- Kolonne 1: luftfugtighed
- Kolonne 2: lufttryk
- Kolonne 3: lys

Skærmen er 8 × 8 pixels, og `sense.set_pixel(x, y, farve)` tænder én ad gangen. Hver måling får sin egen kolonne, og jo højere værdi, jo flere pixels lyser op nedefra. Det var feedback til astronauten.

4. Gem målingerne

linje 286-299

```
with open("team_data.csv", "a") as file:
    file.write("Pressure: %s" % p)
    ...
```

`"a"` betyder *append*: der tilføjes til filen hver gang i stedet for at overskrive den. Samme værdier gemmes også i fire små filer, én pr. sensor.

5. Giv point og vis en smiley

linje 319-333

```
overallstatus = 0
if (15 < t < 20):
    overallstatus = overallstatus + 2   # perfekt temperatur
if (12 < t < 15):
    overallstatus = overallstatus + 1   # god temperatur
...
```

Hver måling giver 2 point, hvis den er perfekt for en kartoffel, og 1 point, hvis den er god. Summen afgør, om skærmen viser en grøn, gul, orange eller rød smiley. Søjlerne var svære at forstå uden forklaring, så holdet lavede smileys, som astronauten kunne aflæse med et blik.

Opgave 2 **Forstå koden**

1. Hvorfor trækker koden 12 fra temperaturen? Hvordan kunne man måle bedre?
2. Kameraet måler lys i én pixel. Hvad er problemet med det? Foreslå en bedre metode.
3. Hvad betyder `"a"` i `open("team_data.csv", "a")`? Hvad ville `"w"` gøre?
4. Hvor mange point kan en måling højst give? Skriv en tabel over, hvilke værdier der giver 2, 1 og 0 point.
5. Tegn et rutediagram over løkken med de fem trin.

04

Fejljagt

Koden blev skrevet af to elever, der lige havde lært at programmere, og på en deadline. Den har fejl. Nogle får programmet til at gå ned, andre giver forkerte data, uden at man opdager det. Alligevel kom den ud i rummet, så fejl er ikke en grund til at lade være.

Opgave 3 **Find mindst fire fejl**

- Hvad sker der, hvis lysværdien er præcis `10` eller `25`? Kig på `if`-sætningerne i linje 68–79.
- Åbn `temp_data.csv`. Kan man læse tallene?
- Tæl, hvor mange gange trykket giver point. Hvad er den højeste score, der kan nås, og hvad viser skærmen så?
- Hvornår når programmet linjen `print("Program ended.")`?
- Bonus: Find variabler, der bliver lavet, men aldrig brugt.

Facit til læreren

- **Huller i grænserne.** `5 < x < 10` efterfulgt af `10 < x < 25` dækker ikke præcis 10. Rammer første måling et hul, findes `status` ikke, og programmet stopper med `NameError`. Løsning: `if/elif/else` med `⩽`.
- **Tal uden adskillelse.** De fire små filer skrives uden linjeskift, så `21.3` og `21.4` bliver til `21.321.4`. `team_data.csv` er heller ikke rigtig CSV, men tekst med etiketter.
- **Trykket tæller dobbelt.** `1005 < p < 1025` giver både 2 og 1 point. Max-scoren bliver 9, men smileyerne dækker kun 0–7, så ved perfekte forhold vises ingen smiley.
- **Uendelig løkke.** `while Update:` sættes aldrig til `False`, så slutningen nås aldrig. Mission Space Lab kræver i dag, at programmet selv stopper inden 10 minutter.
- **Ubrugte variabler:** `selection`, `catalog`, `width`, `height` (bortset fra rettelsen i opgave 1) og `w`, `b`.
- **Gentaget kode.** Søjlerne er skrevet ud pixel for pixel, ca. 200 linjer. En funktion med en løkke kan gøre det samme på 6.

05

Klar til rummet: 2026-versionen

Reglerne for Mission Space Lab har ændret sig siden 2018. Den vigtigste forskel: programmet må ikke længere vise noget til astronauterne. Det er dataopsamlingen, der tæller.

Regel 2026/27	Hvad det betyder for koden
Maks. 10 minutter	Programmet holder selv øje med tiden og stopper i god tid
Ingen interaktion med astronauter	Ingen søjler eller smileys på LED-skærmen
Data ved siden af `main.py`	Brug `Path(__file__).parent`, ingen nye mapper
Maks. 42 billeder ved slut	Tag færre billeder, eller slet dem, I ikke skal bruge
Ingen fejl, der stopper programmet	Fang fejl, så én dårlig måling ikke ødelægger resten

main.py: skabelon til Space Gardeners 2026

```
from sense_hat import SenseHat
from picamzero import Camera
from PIL import Image, ImageStat
from pathlib import Path
from datetime import datetime, timedelta
from time import sleep
import csv

mappe = Path(__file__).parent
sense = SenseHat()
kamera = Camera()

start = datetime.now()
slut = start + timedelta(minutes=9)   # god margin til 10-minutters-grænsen
INTERVAL = 15                         # sekunder, giver ca. 36 billeder


def lysniveau(fil):
    """Gennemsnitlig lysstyrke for hele billedet, 0-100 %."""
    billede = Image.open(fil).convert("L")
    return round(ImageStat.Stat(billede).mean[0] / 255 * 100, 1)


def point(vaerdi, perfekt, god):
    """2 point i det perfekte interval, 1 i det gode, ellers 0."""
    if perfekt[0] ⩽ vaerdi ⩽ perfekt[1]:
        return 2
    if god[0] ⩽ vaerdi ⩽ god[1]:
        return 1
    return 0


with open(mappe / "data.csv", "w", newline="") as f:
    skriver = csv.writer(f)
    skriver.writerow(["tid", "temperatur", "fugtighed", "tryk", "lys", "score"])
```

```
    nr = 0
    while datetime.now() < slut:
        try:
            t = round(sense.get_temperature(), 1)
            h = round(sense.get_humidity(), 1)
            p = round(sense.get_pressure(), 1)

            fil = mappe / f"billede_{nr:03d}.jpg"
            kamera.take_photo(str(fil))
            lys = lysniveau(fil)

            score = (point(t, (15, 20), (12, 23))
                     + point(h, (80, 95), (70, 100))
                     + point(p, (1005, 1025), (990, 1035))
                     + point(lys, (50, 75), (40, 100)))

            skriver.writerow([datetime.now().isoformat(), t, h, p, lys, score])
            f.flush()   # gem med det samme, hvis noget går galt senere
            nr += 1
        except Exception as fejl:
            print("Fejl i måling:", fejl)

        sleep(INTERVAL)

print("Færdig:", nr, "målinger gemt i data.csv")
```

TJEK FØR INDSENDELSE

Hvilke biblioteker, der må bruges på ISS, står i Mission Space Lab Creator Guide — https://rpf.io/msl-creators. Test altid den endelige `main.py` i Replay Online — https://astro-pi-replay-online.astro-pi.org/, før I sender den ind.

Opgave 4 **Gør den til jeres egen**

1. Kør skabelonen og åbn `data.csv` i et regneark. Lav en graf over temperaturen.
2. Sammenlign `lysniveau()` med den gamle metode med én pixel. Hvorfor er den nye bedre?
3. Find ud af, hvilke intervaller der passer bedst for en kartoffel, og ret tallene i `score`.
4. Har I en Sense HAT v2? Prøv at måle lys med `sense.colour.clear` i stedet for kameraet.
5. Fjern billederne efter målingen, så I ikke kommer over 42. Hvilken linje skal tilføjes?

06

Jeres egen mission

Space Gardeners startede med et spørgsmål, som to elever syntes var spændende. Nu er det jeres tur. Mission Space Lab 2026/27 lader jer igen formulere jeres eget projekt, så længe det bruger mindst én sensor eller kameraet og kører i højst 10 minutter.

Opgave 5 **Åben opgave**

1. **Spørgsmålet.** Hvad vil I gerne vide om livet på ISS eller om Jorden set fra rummet? Skriv det som ét spørgsmål.
2. **Målingen.** Hvilke sensorer eller billeder kan svare på det? Sense HAT'en kan måle temperatur, fugt, tryk, acceleration, magnetfelt og lys. Kameraet ser Jorden gennem et vindue.
3. **Planen.** Hvor tit måler I, hvad gemmer I, og hvordan bliver I under 10 minutter og 42 billeder?
4. **Koden.** Byg videre på skabelonen fra opgave 4 og test i Replay Online.
5. **Pitch.** Fremlæg idéen for klassen på 2 minutter: spørgsmål, måling og hvad I forventer at finde.

IDÉER AT STARTE FRA

- Ændrer magnetfeltet sig, når ISS flyver over polerne?
- Kan man se forskel på skov, ørken og hav i infrarøde billeder (NDVI)?
- Hvor hurtigt flyver ISS? Brug to billeder og fælles punkter.
- Hvilke andre afgrøder end kartofler kunne gro i ISS' klima?

07

Til læreren

Modul	Indhold	Tid
1	Projektet, opsætning, opgave 1	45 min
2	Opgave 2 og 3: forstå koden og fejljagt	45 min
3	Opgave 4: 2026-skabelonen og data i regneark	45 min
Forløb	Opgave 5: egen mission, frem mod indsendelse	efter behov

TIDSPLAN FOR MISSION SPACE LAB 2026/27

Deadline for indsendelse	22. februar 2027 kl. 12
Svar om flyvetid	slut april 2027
Programmet kører på ISS	april–maj 2027
Data tilbage	juni 2027

Hold på 2–6 elever i alderen 12–19 år. En lærer kan være mentor for op til 10 hold.

LINKS

- Mission Space Lab: regler 2026/27 — https://astro-pi.org/mission-space-lab/guidelines
- Astro Pi Replay Online — https://astro-pi-replay-online.astro-pi.org/
- Mission B: ISS' hastighed (Rune Klarskov Jensen) — https://rklarskovjensen.github.io/AstroPi2026/
- Raspberry Pi: guide til ISS-hastighed — https://projects.raspberrypi.org/en/projects/astropi-iss-speed/0

Appendiks A

Udstyr og opsætning

Brug dette appendiks, hvis jeres Raspberry Pi ikke er sat op endnu. Når I er igennem trinene, har I en Astro Pi på bordet: en Raspberry Pi med Sense HAT og kamera, klar til at køre koden i afsnit 02.

A.1 Det skal I bruge

Del	Bemærkning
Raspberry Pi 4 eller 5	Én pr. gruppe på 2–3 elever er nok
Sense HAT	Version 2 har også en lys- og farvesensor
Raspberry Pi kamera	Camera Module 2 eller 3, med fladkabel
microSD-kort	Mindst 16 GB
Strømforsyning	Den officielle USB-C-forsyning (Pi 5 kræver 27 W)
Skærm og kabel	Micro-HDMI til HDMI
Tastatur og mus	Almindelige USB-modeller

> INGEN HARDWARE?
>
> Så kan I teste dataopsamlingen i Astro Pi Replay Online — https://astro-pi-replay-online.astro-pi.org/. Den afspiller rigtige data fra ISS, så jeres kode kører, som om den var deroppe. LED-skærmen findes ikke i Replay.

A.2 Opsætning trin for trin

1. **Gør SD-kortet klar**

   Hent Raspberry Pi Imager — https://www.raspberrypi.com/software/ på en almindelig computer. Vælg jeres Pi-model, *Raspberry Pi OS (64-bit)* med skrivebord, og skriv det til SD-kortet.

2. **Monter Sense HAT og kamera, mens strømmen er slukket**

   Sæt Sense HAT'en ned over de 40 GPIO-ben. Sæt kameraets fladkabel i kamerastikket med kontakterne vendt rigtigt. Tryk låsen helt i.

3. **Tilslut og start**

   SD-kort i, så skærm, tastatur og mus, og til sidst strøm. Følg opsætningsguiden første gang (sprog, wifi, brugernavn).

4 Opdater og installer biblioteker

Åbn *Terminal* og kør:

```
terminal

sudo apt update
sudo apt install -y sense-hat python3-picamzero
```

Er pakkerne allerede installeret, siger den bare det.

5 Test Sense HAT

```
terminal

python3 -c "from sense_hat import SenseHat; SenseHat().show_message('Hej ISS')"
```

Teksten skal rulle hen over LED-skærmen.

6 Test kameraet

```
terminal

rpicam-hello -t 5000
```

Et preview-vindue skal vise kamerabilledet i 5 sekunder. Sker der ingenting, så sluk og tjek fladkablet.

7 Åbn Thonny

Thonny ligger under *Programmering* i menuen. Det er editoren, I skriver og kører koden i.

Udarbejdet til Sciencekonferencen, Thornbjerg Gymnasium, 13. november 2026.

Rune Klarskov Jensen · Mohammad Ahmad · Frederik Paustian